Physics-Informed Deep Learning for Wave-Source Localization

Assaf Zimand

September 16, 2025

Abstract

This project was completed as part of a course on the mathematical foundations of machine learning. The task was to build a deep learning pipeline that can localize the source of a 2D wave field. The goal is to connect wave physics with modern deep learning and to examine how internal layers behave on physics-related problems using interpretation tools such as feature extraction and activation maximization.
The best model achieved a mean localization error of 1.66 pixels (1.3% of the image width). Comparable performance was obtained in the other simulation regime.

1 Problem and Data

Task: given a 128×128 wave-field snapshot, predict the source coordinates (x, y). The datasets were generated specifically for this project. Using a numerical wave simulator (a finite-difference time-domain solver of the 2D wave equation with homogeneous wave speed and perfectly reflecting (Neumann) boundary conditions), a point source was chosen at a random grid location and the resulting field at time step T was recorded. Each dataset is stored in HDF5 format and contains:

- `wave_fields`: $(N, 128, 128)$ arrays of simulated waves.
- `source_coords`: $(N, 2)$ arrays of true source positions.
- `wave_mean`, `wave_std` for normalization.

There are two simulation regimes (The notation T=250 and T=500 is used consistently in this report):

- **T=250**: shorter simulation length (250 steps).
- **T=500**: longer simulation length (500 steps).

To ensure consistency, models trained on T = 500 data are evaluated only on T=500 data (analogously for T=250), using the corresponding normalization parameters.

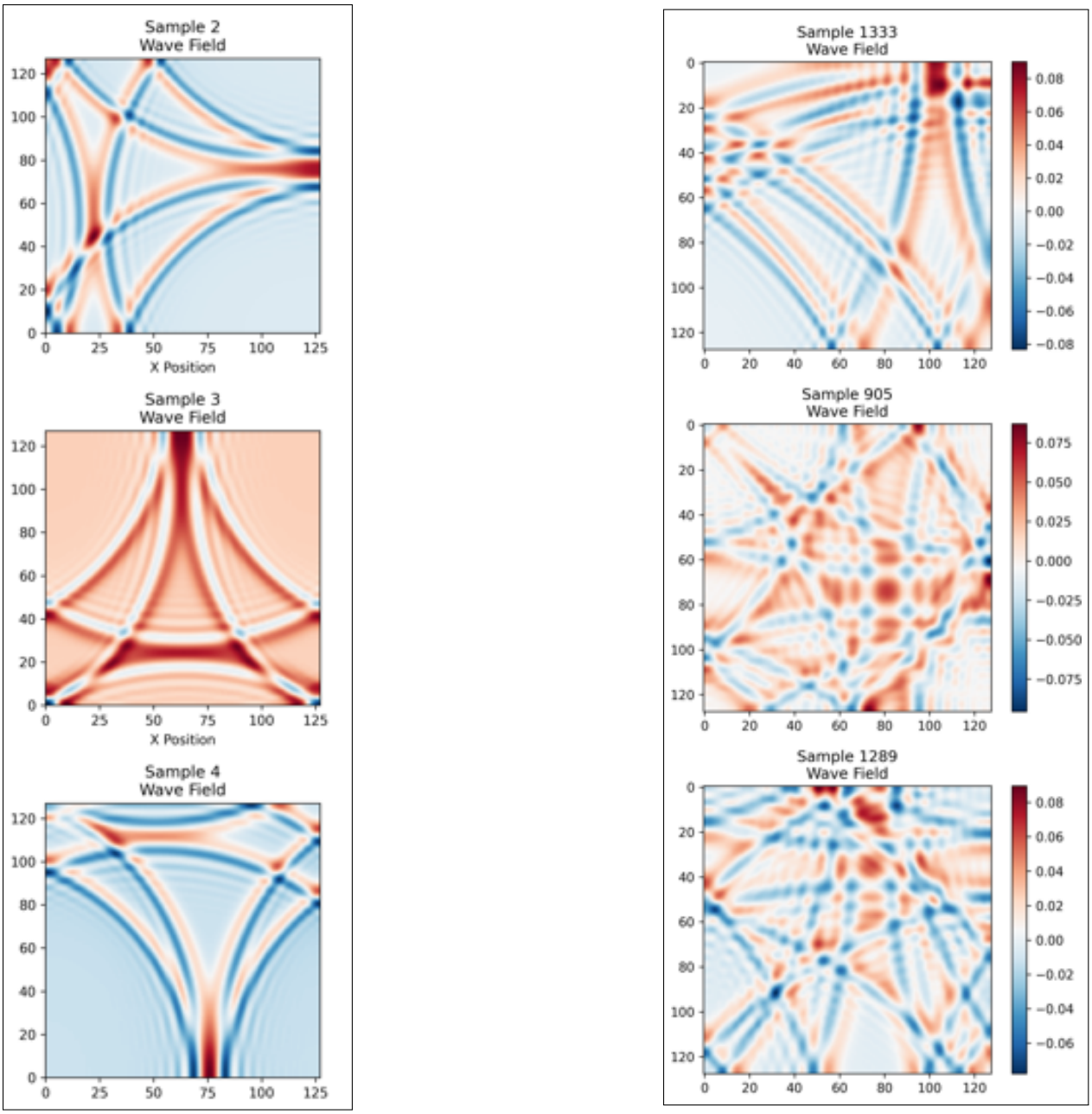


Figure 1: T=250 (left) and T=500 (right) data samples.

2 Model

The chosen model is a compact residual network, **WaveSourceMiniResNet**. The approach is to start small while retaining the main strengths of ResNet: skip connections, stability in training, and good gradient flow. This model produced strong early results; therefore, alternative architectures were not pursued.

For a compact architectural description, see Appendix B; a schematic visualization is available at model_visualization.

3 Training and Evaluation

Evaluation metric across all runs was the Euclidean error in pixels ($\sqrt{(\hat{x}-x)^2+(\hat{y}-y)^2}$). Statistical parameters and error distributions were reported where appropriate.
Training proceeded in several stages:

1. Local proof-of-concept runs to make sure everything worked.
2. Google Colab (GPU) to run larger experiments efficiently, including grid search and 5-fold cross-validation (CV).
3. For grid search the following hyperparameters were explored: learning rate $[1e{-}3, 1e{-}4]$, batch size$[16, 32]$, optimizer $[Adam/AdamW]$, and the number of training epochs (increased until no further validation improvement was observed). Model selection used the lowest mean validation error.

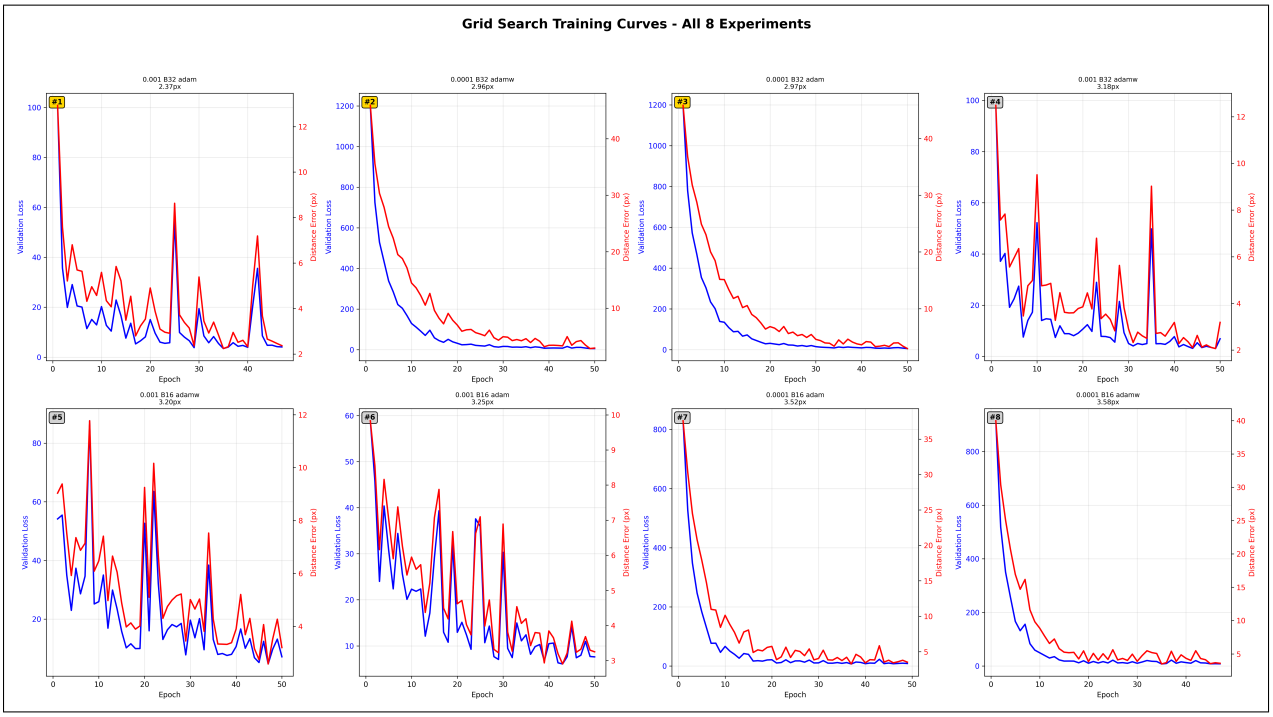


Figure 2: T=500 grid search training curves.

Grid Search Results Summary

	Experiment	Learning Rate	Batch Size	Optimizer	Distance Error (px)	Validation Loss
1	**lr0.001_bs32_adam**	**0.001**	**32**	**adam**	**2.37**	**4.0676**
2	**lr0.0001_bs32_adamw**	**0.0001**	**32**	**adamw**	**2.96**	**5.6514**
3	**lr0.0001_bs32_adam**	**0.0001**	**32**	**adam**	**2.97**	**5.9881**
4	lr0.001_bs32_adamw	0.001	32	adamw	3.18	6.8796
5	lr0.001_bs16_adamw	0.001	16	adamw	3.20	7.1394
6	lr0.001_bs16_adam	0.001	16	adam	3.25	7.6099
7	lr0.0001_bs16_adam	0.0001	16	adam	3.52	8.5347
8	lr0.0001_bs16_adamw	0.0001	16	adamw	3.58	8.3263

Figure 3: T=500 grid search summary table.

4. For CV, training ran for 75 epochs per fold with early stopping (patience 15), weight decay 0.01, and He initialization; random seeds were fixed for reproducibility. Computation used a Colab GPU (L4); time per epoch ∼1.6 min and total training time ∼10 hours for 5 folds. Folds were formed by random partitioning at the sample level. Model selection used the lowest mean Euclidean error on the validation set; optionally, a final model can be retrained on the union of training folds.

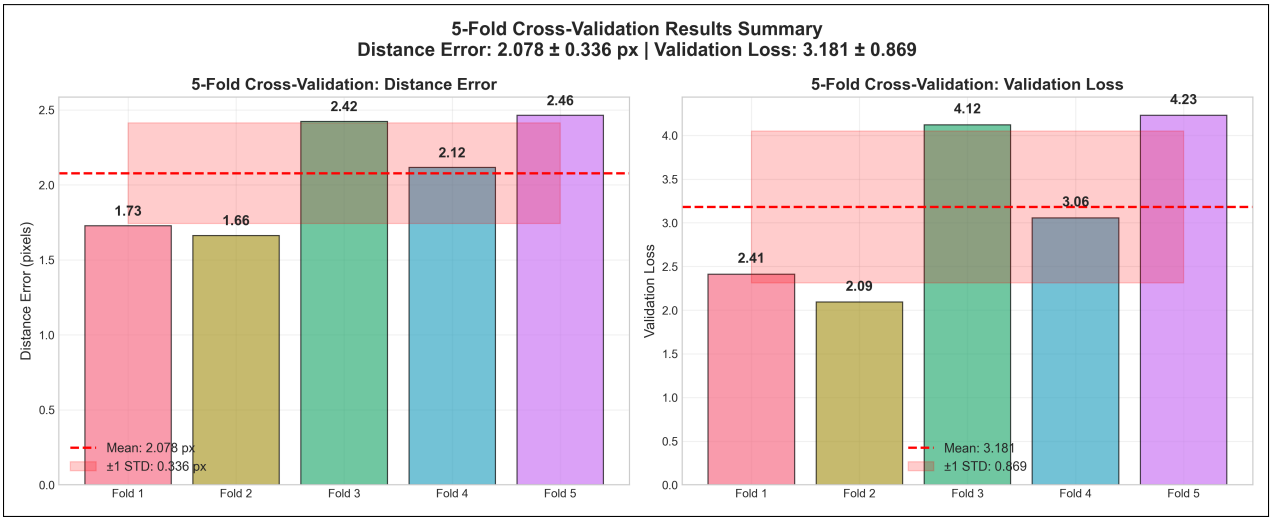
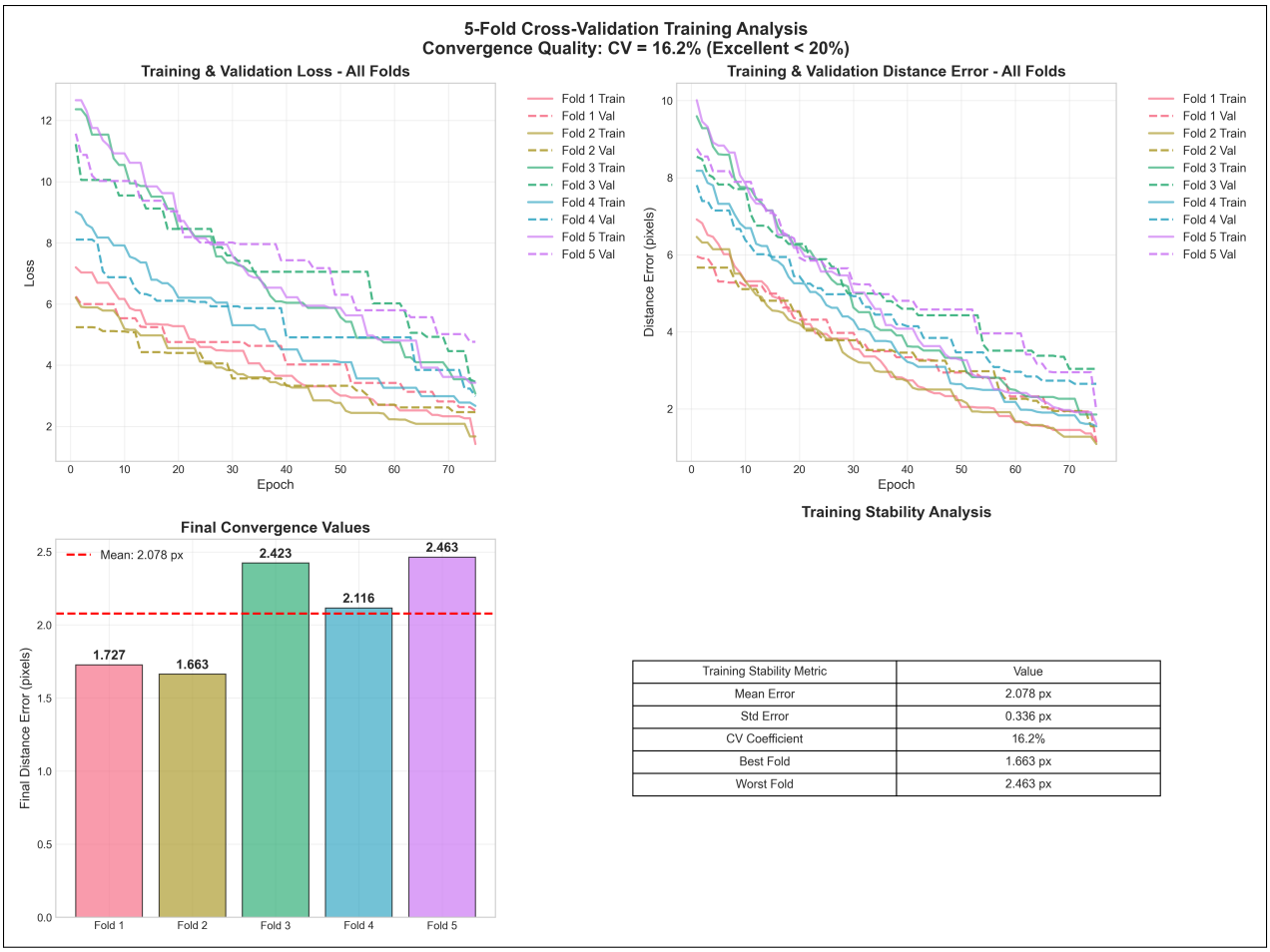


Figure 4: T=500 cross-validation results summary. Metric: Euclidean localization error (pixels).



Training Stability Metric	Value
Mean Error	2.078 px
Std Error	0.336 px
CV Coefficient	16.2%
Best Fold	1.663 px
Worst Fold	2.463 px

Figure 5: T=500 training curves across five folds. Metric: Euclidean localization error (pixels).

5. A separate T=250 pipeline was built to demonstrate that the method generalizes across different temporal regimes (T=250 vs T=500).

6. Extra validation was performed on newly generated, unseen datasets (the simulator can produce unlimited samples), to test generalization and show statistical significance.

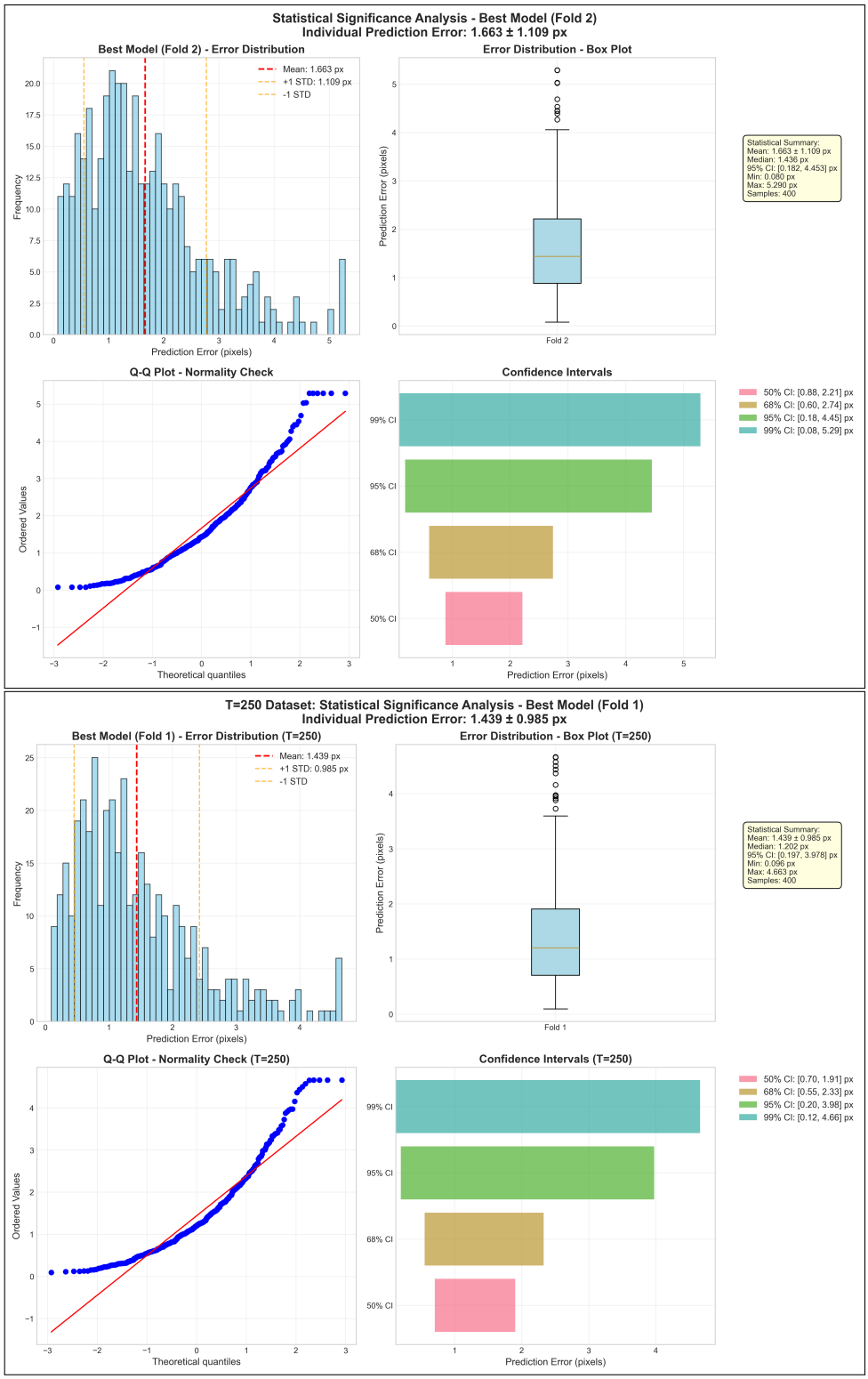


Figure 6: Validation analysis for both T=500 and T=250. Plots show distributions of Euclidean error (pixels) with statistical parameters and confidence intervals.

4 Failure Analysis

A detailed error analysis was conducted to understand the model's predictions and limitations.

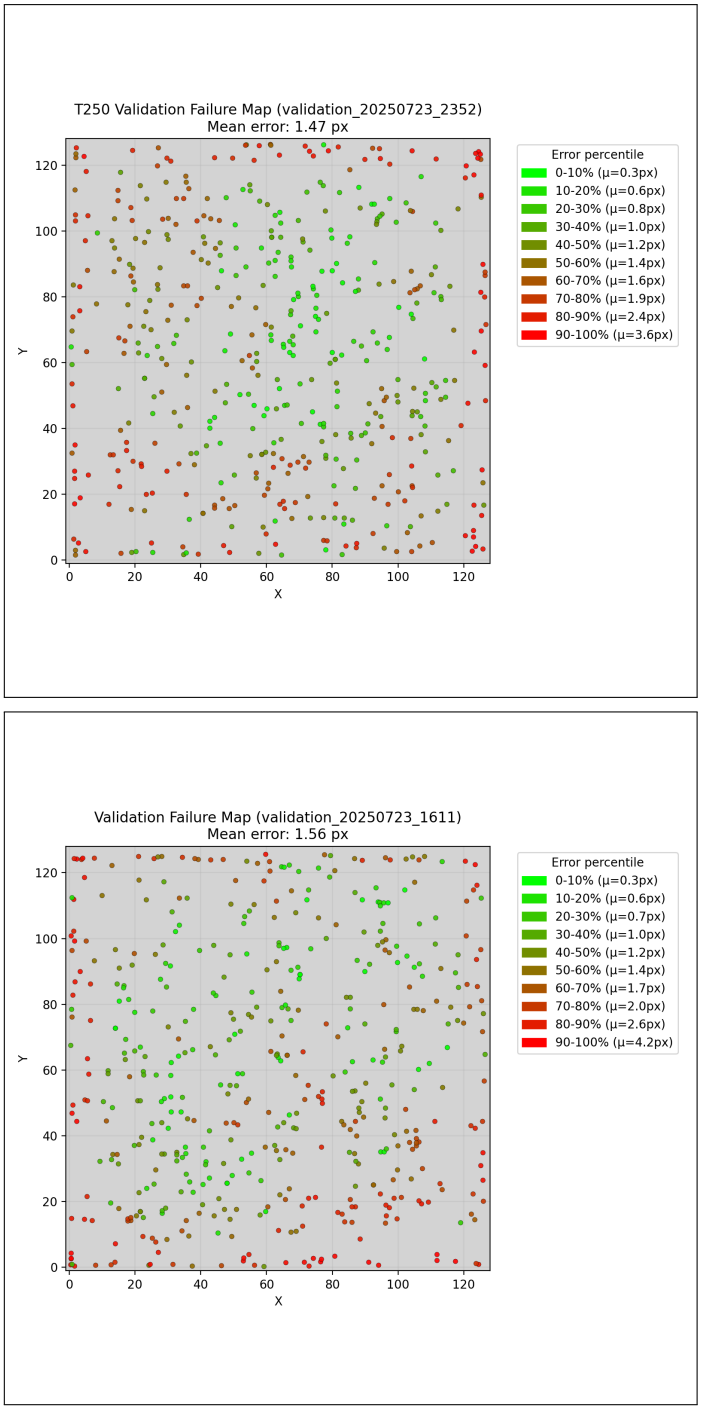


Figure 7: Both T=250 and T=500 error maps.

These plots indicate that larger errors typically occur near image boundaries. This suggests the task is more difficult near boundaries, and that the model may benefit from proportionally more training examples in these regions relative to mid-grid points.

5 Interpreting the Model

5.1 Feature Extraction Analysis

To better understand model behavior, we first examined feature extraction. The following plots show feature maps from the most-activated filters in selected layers for a representative sample. The most active feature maps often highlight regions near the true source, which suggests the network uses cues around the source. This is a qualitative observation and should be tested more rigorously.

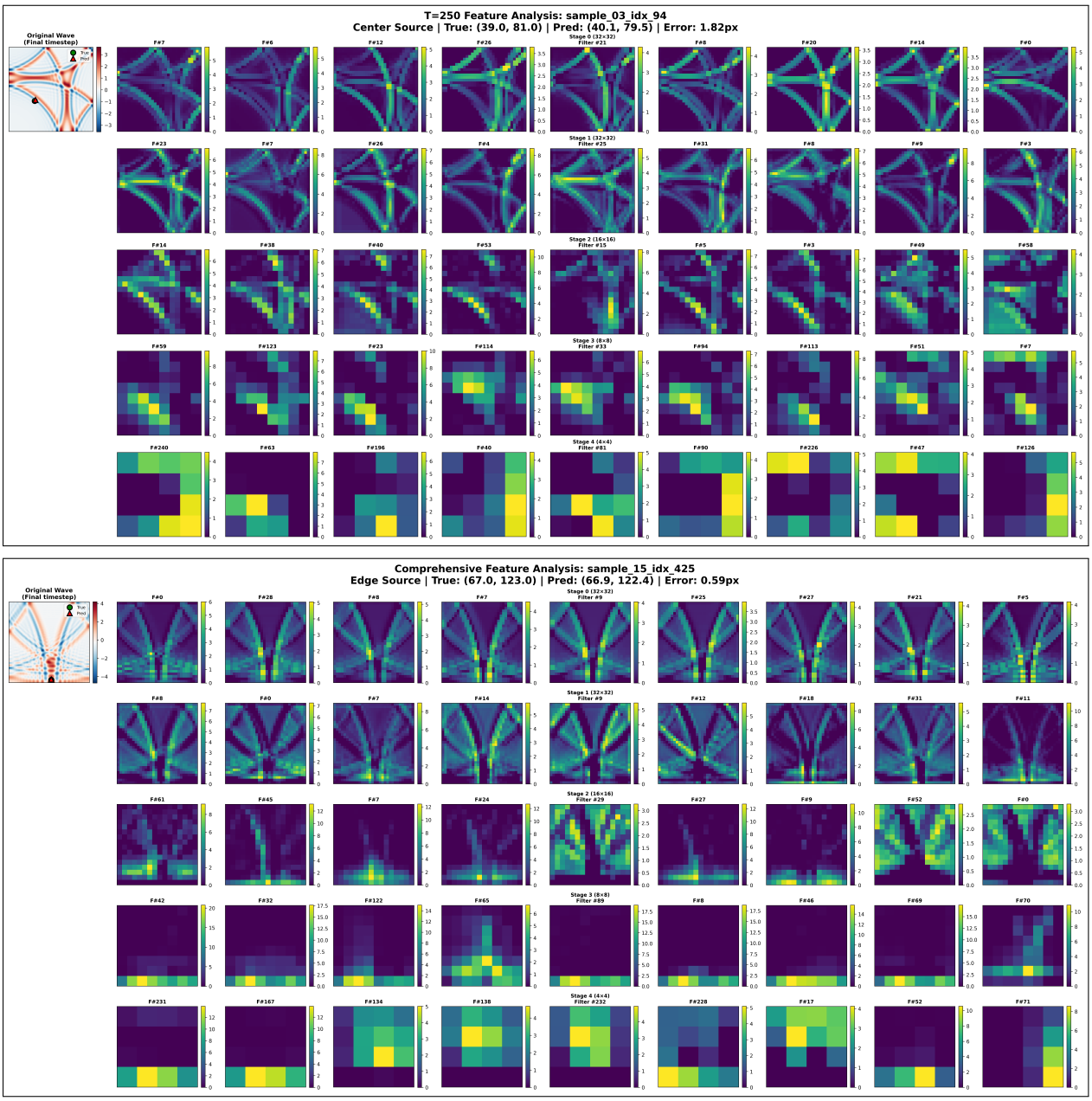


Figure 8: Feature maps on representative T=250/T=500 samples; one image per filter; rows correspond to layers.

5.2 Activation Maximization

A unified pipeline for Activation Maximization (AM) was implemented:

- Pre-ReLU activations of filter outputs are captured using hooks.
- Objective: maximize the activation of a target filter while optionally suppressing other filters in the same layer.
- Motivated by initial results indicating that deeper layers respond to higher-amplitude regions, a comprehensive grid search was performed to better visualize input patterns corresponding to specific filters by varying loss terms and their weights.
  - Regularizers on the input: Total Variation (TV) to promote piecewise smoothness and L2 to control overall energy.
  - Activation measures: pre-ReLU mean, pre-ReLU mean_abs, pre-ReLU L2, and post-ReLU mean.
- Optimization: forward a sample, select filters that are most active in their layer from that sample, then perform Adam steps on the input, starting at the sample, to minimize the total loss.
- Loss function (for layer ℓ, target filter k):

$$L(x) = -A_{\ell,k}(x) + \lambda_{\text{sup}} \sum_{j \neq k} A_{\ell,j}(x) + \lambda_{\text{TV}} \, \text{TV}(x) + \lambda_{\text{L2}} \, \|x\|_2^2$$

  where $A_{\ell,k}(x)$ denotes the (pre/post-ReLU) activation, depending on the chosen measure. The goal is to find $\arg\min_x L(x)$.

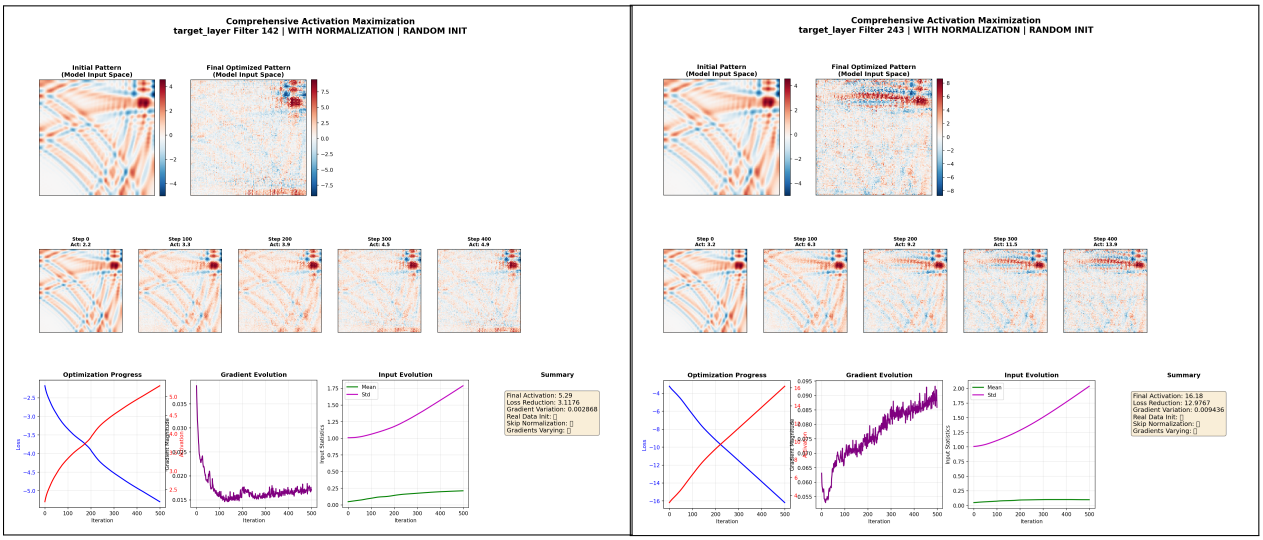


Figure 9: Initial AM results with $(\lambda_{\text{sup}}, \lambda_{\text{TV}}, \lambda_{\text{L2}}) = (0, 0, 0)$. Each image shows the optimized input pattern and the optimization process statistics for a target filter.

Many combinations of activation measures, suppression weighting, and regularization strengths were tested to find a stable and informative objective. post-ReLU mean activation was chosen (similar in results to l2) because it produced the most stable results and aligns with the model's computation (activations propagate post-ReLU across layers). The exploration of the right set of $(\lambda_{\text{sup}}, \lambda_{\text{TV}}, \lambda_{\text{L2}})$ is still an open subject as a single set that produces satisfying results has yet to be found. In the next set of examples, we discuss some of the trade-offs that this set of parameters contains.

- Adjusting λ_{sup} initially got the expected results of cleaning parts of the image that are probably less related to the target filter.

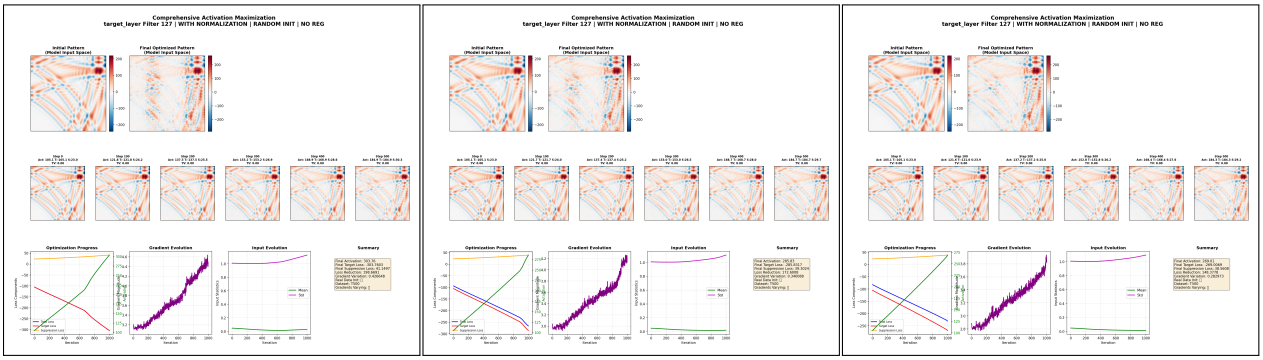


Figure 10: Effect of suppressing other filters ($\lambda_{\text{sup}} = 0/0.5/1$, left to right).

- In the following images, we can see some results of $(\lambda_{\text{sup}}, \lambda_{\text{TV}}, \lambda_{\text{L2}}) = (1, 0, 0)$. With these parameters, input energy is unconstrained, so the objective prioritizes increasing the target filter's activation over suppressing others (as can be seen at the left bottom graph of every image). The resulting patterns are consistent with the initial runs at $\lambda_{\text{sup}} = 0$.

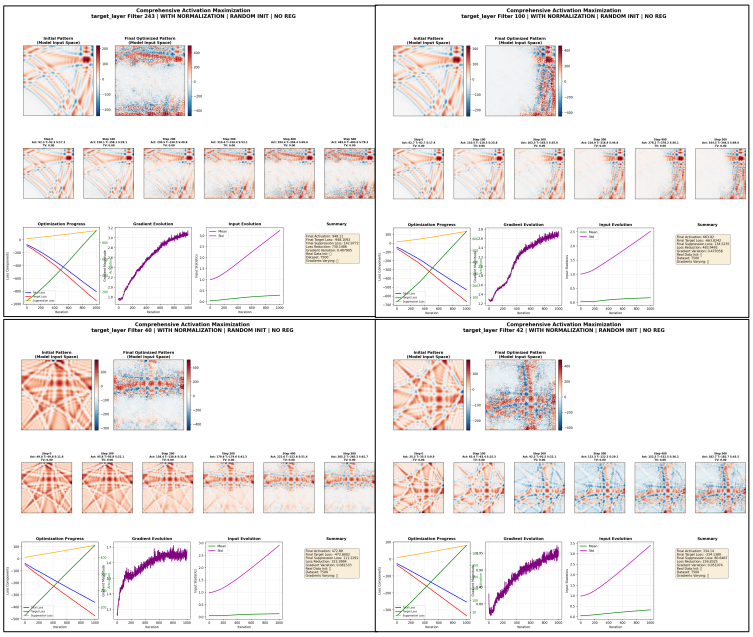


Figure 11: Representative Activation Maximization outcomes (T=500, $(\lambda_{\text{sup}}, \lambda_{\text{TV}}, \lambda_{\text{L2}}) = (1, 0, 0)$).

- As $(\lambda_{\text{sup}}, \lambda_{\text{TV}}, \lambda_{\text{L2}})$ increase we get a cleaner and smoother result that points to a smaller part of the image that the target filter recognizes. But the target filter's activation often decreases. The tipping point varies across samples, layers, and filters. Combined with the lack of a single 'desired' pattern, this variability leaves the AM procedure unsettled.

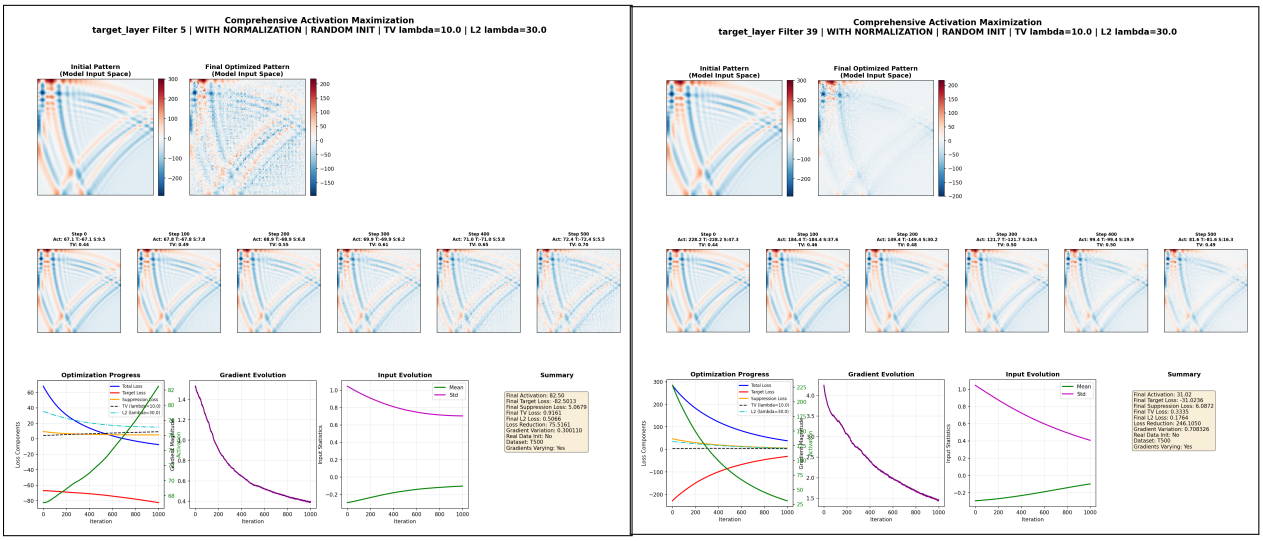


Figure 12: AM results with $(\lambda_{\text{sup}}, \lambda_{\text{TV}}, \lambda_{\text{L2}}) = (10, 10, 30)$. On layers 10, 29, filters are chosen from the same sample. Parameters are the same but the results are very different in their meanings (red graph is the target filter activation).

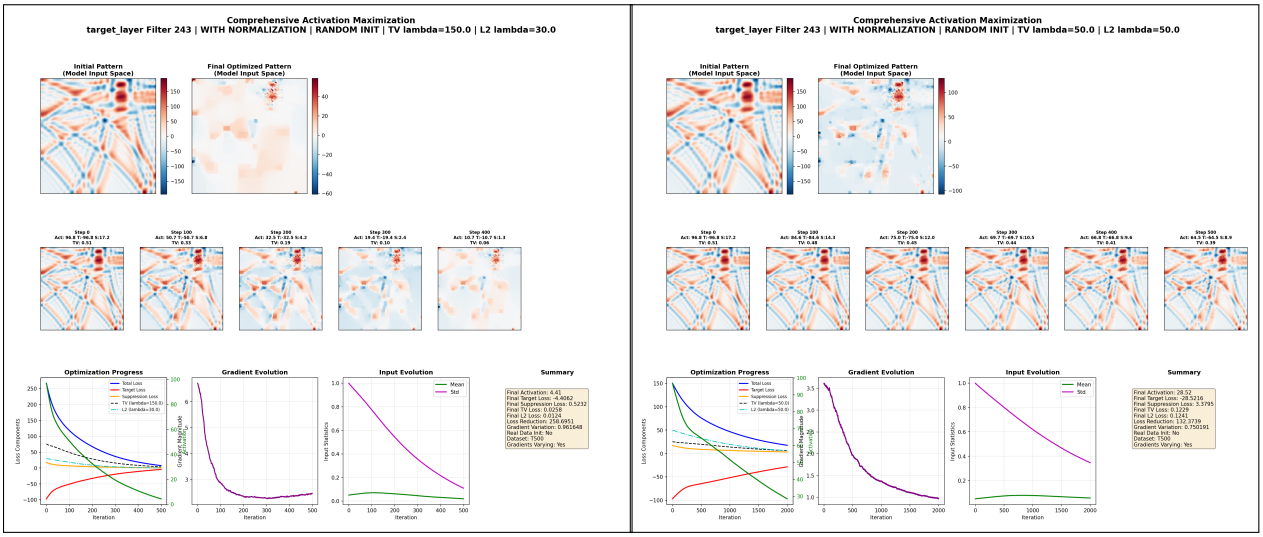


Figure 13: AM results on the same sample, layer and filter with different set of high values of $(\lambda_{\text{sup}}, \lambda_{\text{TV}}, \lambda_{\text{L2}})$. This raises the question: which configuration should be preferred?

6 Lessons Learned and Future Work

- Both grid search and CV reduced error relative to naive baselines.
- Error analysis revealed that borders are the main weakness of the model. This enables targeted improvements, for example, boundary-aware weighting.
- Activation Maximization gave a sense of realization on how to visualize what the network focuses on. Unfortunately results are not yet stable or clear; further investigation is required. Next steps include defining a clearer objective and adding physics-informed regularization (e.g., energy constraints)..
- The start-small approach was applied across all stages, in particular, a compact ResNet-style model proved simple yet sufficiently expressive for this task.
- A demo exists for the reader to view the model's predictions and AM results, further instructions can be found in the GitHub repository.

7 References

- S. Dekel, D. Givoli, A. Kahana, E. Turkel (2020). Obstacle segmentation based on the wave equation and deep learning. Journal of Computational Physics, 413.

Appendices

A Code Repository

The full project code, pretrained models, and demo instructions are available at:
`https://github.com/assafzimand/Physics-Informed-DL-Project`

B Model Architecture

WaveSourceMiniResNet Architecture Summary

Overview

A custom ResNet-based CNN for predicting wave source coordinates from wave interference patterns. The network processes 128×128 wave field images and outputs (x, y) coordinates in the range [0, 127].

Stage 0:

Layer	Input Shape	Output Shape	Filters	Kernel	Stride	Padding	Change
Conv2d	[B,1,128,128]	[B,32,64,64]	32	7×7	2	3	Spatial: 128→64, Ch: 1→32
BatchNorm2d	[B,32,64,64]	[B,32,64,64]	32 ch	-	-	-	No change
ReLU	[B,32,64,64]	[B,32,64,64]	-	-	-	-	No change
MaxPool2d	[B,32,64,64]	[B,32,32,32]	-	3×3	2	1	Spatial: 64→32

Stage 1:

Block 1 (Identity Skip)

Layer	Input Shape	Output Shape	Filters	Kernel	Stride	Padding	Change
Conv1	[B,32,32,32]	[B,32,32,32]	32	3×3	1	1	No change
BN1+ReLU	[B,32,32,32]	[B,32,32,32]	32 ch	-	-	-	No change

Layer	Input Shape	Output Shape	Filters	Kernel	Stride	Padding	Change
Conv2	[B,32,32,32]	[B,32,32,32]	32	3×3	1	1	No change
BN2	[B,32,32,32]	[B,32,32,32]	32 ch	-	-	-	No change
Skip	[B,32,32,32]	[B,32,32,32]	-	-	-	-	Identity
Add+ReLU	[B,32,32,32]	[B,32,32,32]	-	-	-	-	No change

Block 2 (Identity Skip)

Layer	Input Shape	Output Shape	Filters	Kernel	Stride	Padding	Change
Conv1	[B,32,32,32]	[B,32,32,32]	32	3×3	1	1	No change
BN1+ReLU	[B,32,32,32]	[B,32,32,32]	32 ch	-	-	-	No change
Conv2	[B,32,32,32]	[B,32,32,32]	32	3×3	1	1	No change
BN2	[B,32,32,32]	[B,32,32,32]	32 ch	-	-	-	No change
Skip	[B,32,32,32]	[B,32,32,32]	-	-	-	-	Identity
Add+ReLU	[B,32,32,32]	[B,32,32,32]	-	-	-	-	No change

Stage 2:

Block 1 (Projection Skip)

Layer	Input Shape	Output Shape	Filters	Kernel	Stride	Padding	Change
Conv1	[B,32,32,32]	[B,64,16,16]	64	3×3	2	1	Spatial: 32→16, Ch: 32→64
BN1+ReLU	[B,64,16,16]	[B,64,16,16]	64 ch	-	-	-	No change
Conv2	[B,64,16,16]	[B,64,16,16]	64	3×3	1	1	No change

Layer	Input Shape	Output Shape	Filters	Kernel	Stride	Padding	Change
BN2	[B,64,16,16]	[B,64,16,16]	64 ch	-	-	-	No change
Skip	[B,32,32,32]	[B,64,16,16]	64	1×1	2	0	Projection
Add+ReLU	[B,64,16,16]	[B,64,16,16]	-	-	-	-	No change

Block 2 (Identity Skip)

Layer	Input Shape	Output Shape	Filters	Kernel	Stride	Padding	Change
Conv1	[B,64,16,16]	[B,64,16,16]	64	3×3	1	1	No change
BN1+ReLU	[B,64,16,16]	[B,64,16,16]	64 ch	-	-	-	No change
Conv2	[B,64,16,16]	[B,64,16,16]	64	3×3	1	1	No change
BN2	[B,64,16,16]	[B,64,16,16]	64 ch	-	-	-	No change
Skip	[B,64,16,16]	[B,64,16,16]	-	-	-	-	Identity
Add+ReLU	[B,64,16,16]	[B,64,16,16]	-	-	-	-	No change

Stage 3:

Block 1 (Projection Skip)

Layer	Input Shape	Output Shape	Filters	Kernel	Stride	Padding	Change
Conv1	[B,64,16,16]	[B,128,8,8]	128	3×3	2	1	Spatial: 16→8, Ch: 64→128
BN1+ReLU	[B,128,8,8]	[B,128,8,8]	128 ch	-	-	-	No change
Conv2	[B,128,8,8]	[B,128,8,8]	128	3×3	1	1	No change
BN2	[B,128,8,8]	[B,128,8,8]	128 ch	-	-	-	No change
Skip	[B,64,16,16]	[B,128,8,8]	128	1×1	2	0	Projection
Add+ReLU	[B,128,8,8]	[B,128,8,8]	-	-	-	-	No change

Block 2 (Identity Skip)

Layer	Input Shape	Output Shape	Filters	Kernel	Stride	Padding	Change
Conv1	[B,128,8,8]	[B,128,8,8]	128	3×3	1	1	No change
BN1+ReLU	[B,128,8,8]	[B,128,8,8]	128 ch	-	-	-	No change
Conv2	[B,128,8,8]	[B,128,8,8]	128	3×3	1	1	No change
BN2	[B,128,8,8]	[B,128,8,8]	128 ch	-	-	-	No change
Skip	[B,128,8,8]	[B,128,8,8]	-	-	-	-	Identity
Add+ReLU	[B,128,8,8]	[B,128,8,8]	-	-	-	-	No change

Stage 4:

Block 1 (Projection Skip)

Layer	Input Shape	Output Shape	Filters	Kernel	Stride	Padding	Change
Conv1	[B,128,8,8]	[B,256,4,4]	256	3×3	2	1	Spatial: 8→4, Ch: 128→256
BN1+ReLU	[B,256,4,4]	[B,256,4,4]	256 ch	-	-	-	No change
Conv2	[B,256,4,4]	[B,256,4,4]	256	3×3	1	1	No change
BN2	[B,256,4,4]	[B,256,4,4]	256 ch	-	-	-	No change
Skip	[B,128,8,8]	[B,256,4,4]	256	1×1	2	0	Projection
Add+ReLU	[B,256,4,4]	[B,256,4,4]	-	-	-	-	No change

Block 2 (Identity Skip)

Layer	Input Shape	Output Shape	Filters	Kernel	Stride	Padding	Change
Conv1	[B,256,4,4]	[B,256,4,4]	256	3×3	1	1	No change
BN1+ReLU	[B,256,4,4]	[B,256,4,4]	256 ch	-	-	-	No change
Conv2	[B,256,4,4]	[B,256,4,4]	256	3×3	1	1	No change
BN2	[B,256,4,4]	[B,256,4,4]	256 ch	-	-	-	No change
Skip	[B,256,4,4]	[B,256,4,4]	256	-	-	-	Identity

Layer	Input Shape	Output Shape	Filters	Kernel	Stride	Padding	Change
Add+ReLU	[B,256,4,4]	[B,256,4,4]	-	-	-	-	No change

Global Pooling & Regression Head

Layer	Input Shape	Output Shape	Neurons	Operation	Change
AdaptiveAvgPool2d	[B,256,4,4]	[B,256,1,1]	-	4×4→1×1	Spatial collapse
Flatten	[B,256,1,1]	[B,256]	-	Reshape	4D→2D
Linear	[B,256]	[B,128]	256→128	FC layer	Features: 256→128
BN1d+ReLU+Drop(0.2)	[B,128]	[B,128]	128	Regularize	20% dropout
Linear	[B,128]	[B,64]	128→64	FC layer	Features: 128→64
BN1d+ReLU+Drop(0.1)	[B,64]	[B,64]	64	Regularize	10% dropout
Linear	[B,64]	[B,2]	64→2	FC layer	Final coordinates
Sigmoid×127	[B,2]	[B,2]	-	Scale	Range [0,127]

Key Statistics

Metric	Value
Total Convolution Filters	1,952 filters
Total Parameters	~1.2M trainable parameters
Spatial Reduction	128×128 → 4×4 → 1×1 (16,384× reduction)
Channel Expansion	1 → 32 → 64 → 128 → 256 (256× expansion)
Final Output	2 coordinates in range [0, 127]

Architecture Components

Residual Blocks

- **Structure**: Two 3×3 convolutions with batch normalization
- **Skip Connections**:
  - Identity when input/output shapes match
  - 1×1 projection convolution when shapes differ
- **Pattern**: First block changes dimensions, second block refines features

Skip Connection Types

Type	When Used	Operation
Identity	Same input/output shape	Direct addition
Projection	Different input/output shape	1×1 conv + stride to match dimensions

Dimension Progression

Stage	Spatial Size	Channels	Feature Maps
Input	128×128	1	1
Stage 0	32×32	32	32
Stage 1	32×32	32	32
Stage 2	16×16	64	64
Stage 3	8×8	128	128
Stage 4	4×4	256	256
Global Pool	1×1	256	256
Output	-	2	2 coordinates

Technical Details

Convolution Parameters

- **Initial Conv**: 7×7 kernel, stride=2 for rapid spatial reduction
- **Residual Convs**: 3×3 kernels, stride=1 or 2 depending on block type
- **Projection Convs**: 1×1 kernels for efficient channel transformation

Regularization

- **Batch Normalization**: Applied after every convolution

- **Dropout**: 0.2 and 0.1 rates in regression head
- **ReLU Activation**: Applied after batch normalization

Global Pooling

- **AdaptiveAvgPool2d(1)**: Converts 4×4 feature maps to single values
- **Purpose**: Translation invariance and dimension reduction

Output Processing

- **Sigmoid Activation**: Ensures output in [0, 1] range
- **Scaling**: Multiply by 127 to get coordinates in [0, 127] range

---

Generated from WaveSourceMiniResNet architecture analysis